

Lecture 12: Introduction to Process Scheduling

Vivek Kumar

Computer Science and Engineering

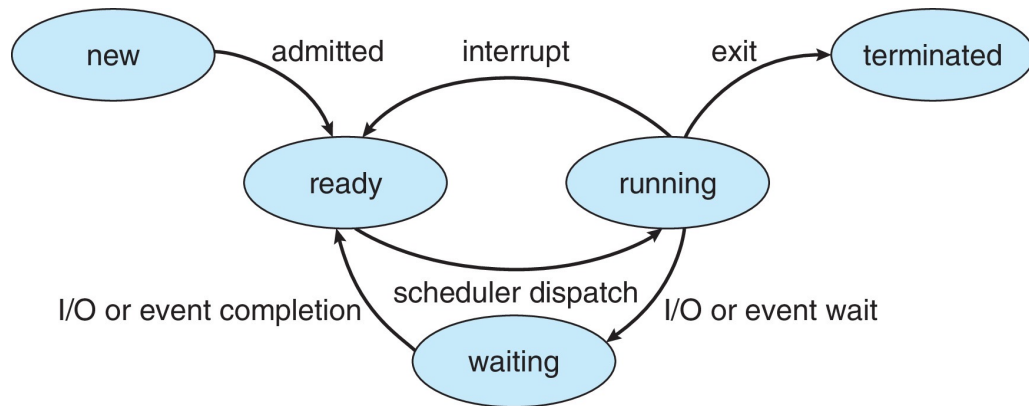
IIIT Delhi

vivekk@iiitd.ac.in

Today's Class

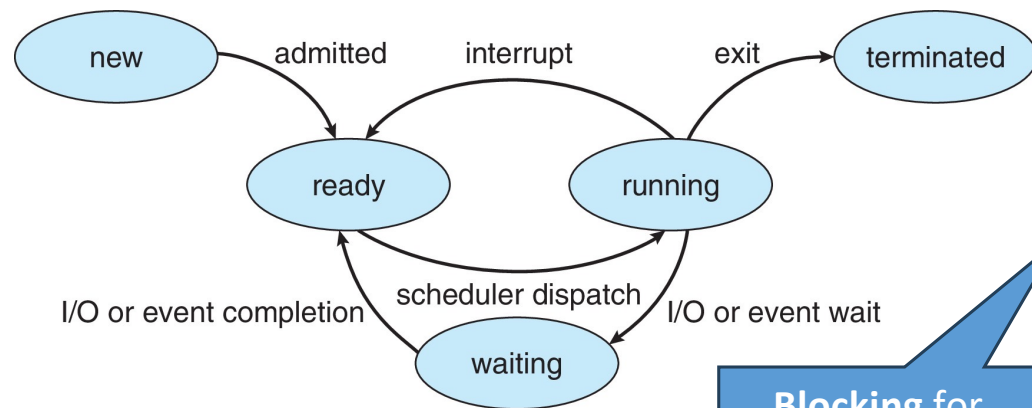
- Introduction to process scheduling
 - Context switch
 - Process scheduling policies

Process State in these Scenarios?



1. Write on pipe that is currently full
2. `scanf()` waiting for user input
3. Recursive Fibonacci number calculation
4. Process executing `sem_wait` when
 - Semaphore > 0
5. Process executing `sem_wait` when
 - Semaphore ≤ 0
6. Process executing `sem_post`
7. `sleep()`
8. `wait()`

Process State in these Scenarios?



Blocking for
some resources

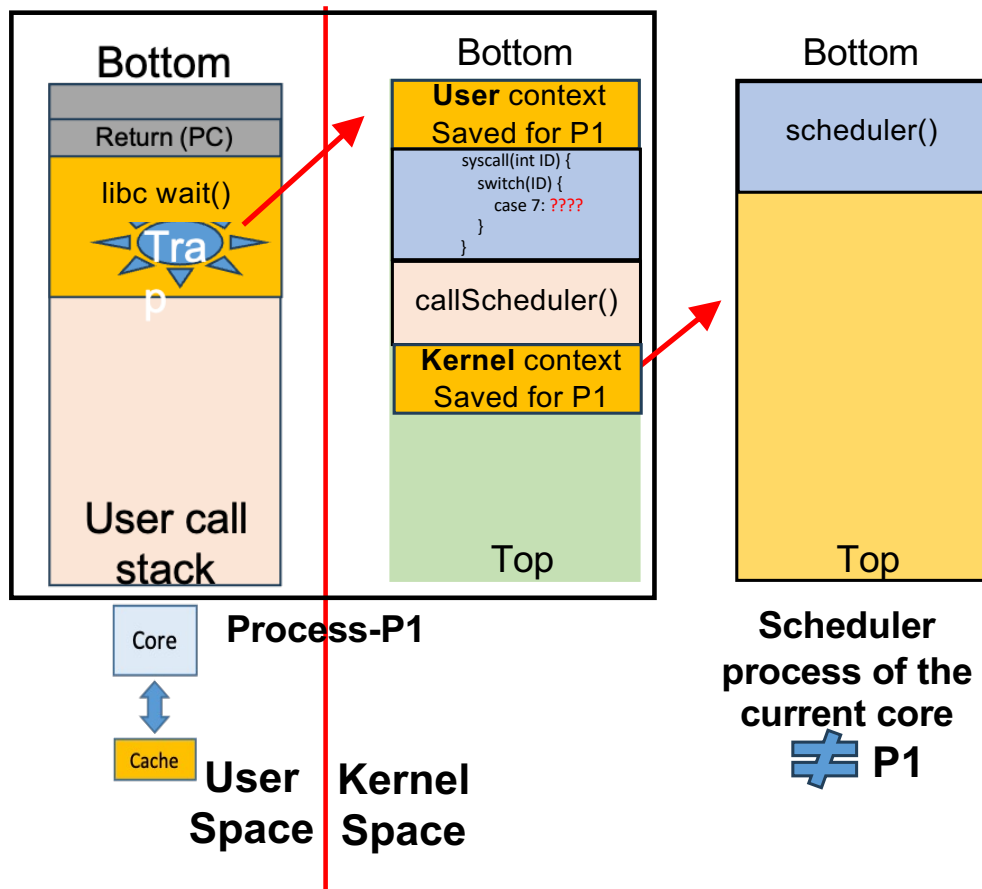
Not Blocking –
Running until
termination

1. Write on pipe that is currently full
2. scanf() waiting for user input
3. sleep()
4. wait()
5. Process executing sem_wait when
 - Semaphore ≤ 0
6. Recursive Fibonacci number calculation
7. Process executing sem_wait when
 - Semaphore > 0
8. Process executing sem_post

How OS Can Juggle Between Processes?

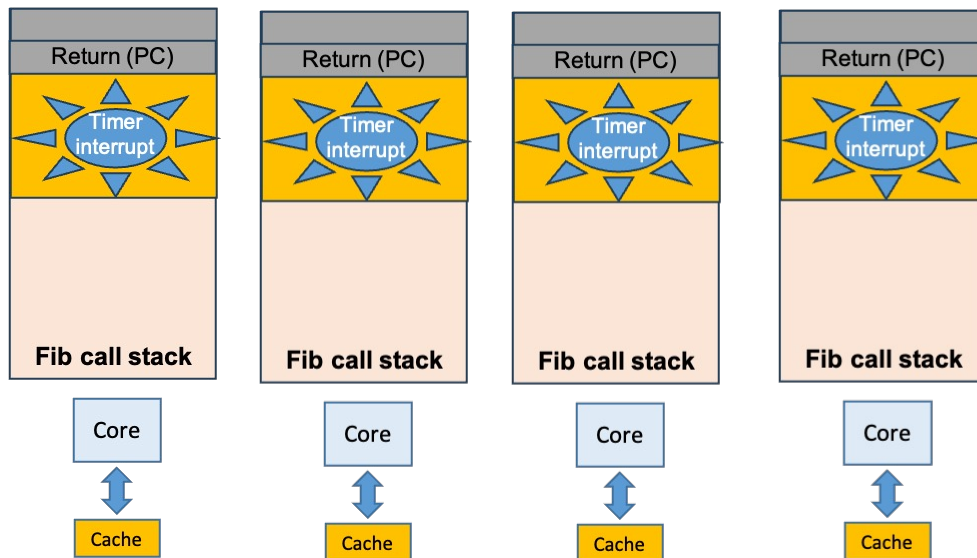
- Case-1: Whenever a process is **blocking** for some resource
 - It does not make any sense to let a blocked process running on the CPU → wastage of CPU cycles
 - Even if the CPUs are idle
 - OS can safely move the process from the running to waiting queue
 - Does not affect the performance, as the process can be brought back into the running queue as soon as it unblocks
- Case-2: If a process is **NOT blocking**
 - Timer interrupt is generated by the CPU at **fixed** interval (hardware interrupt)
 - It is used to trigger the scheduling by transferring control to the OS scheduler, which moves the currently running process into the ready queue and another process from the ready queue into the running queue

Case-1: Process is Going to Block



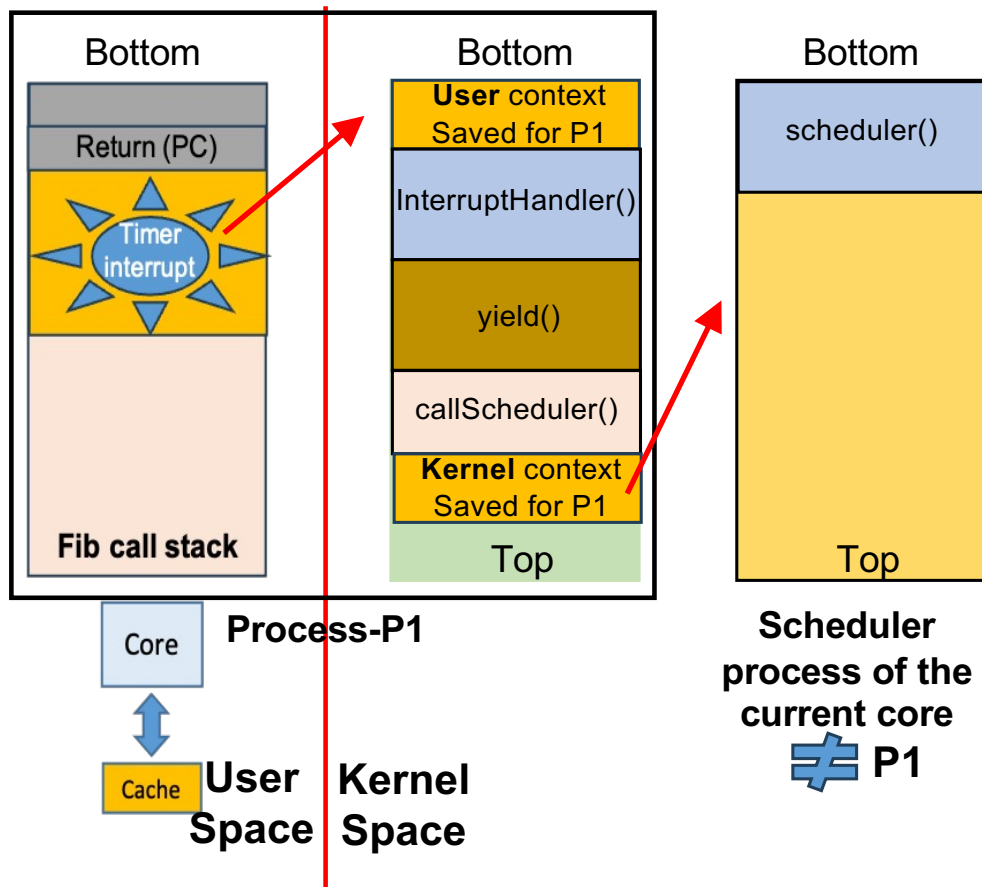
- The user process P1 calls a blocking API (e.g., **waitpid()**)
 - We have already gone through its working in Lecture 06
 - But what happens inside the switch-case block of syscall handler for sleep?
- The **waitpid** will invoke a call to the CPU scheduler, which will: a) save the kernel context of the user process at the top of the kernel stack, b) load the context of the CPU scheduler from the corresponding stack, and c) jump to the CPU scheduler stack where it will call the **schedule()** routine
 - Steps “a” and “b” are referred to as **context switch**
- Each CPU has its own CPU scheduler process that runs on a separate kernel stack than that of the process it was originally executing (**Why?**)

Case-2: Process is Running until Termination



- There could be several Fibonacci processes being displayed in **running** state from htop on a 4-core processor
 - However, only four of them are **actually** in running state any given time (1-1 mapping between a process and a core)
- Timer interrupt (IDT index 32 on x86) is generated by the hardware after every fixed intervals

Case-2: Process is Not Going to Block



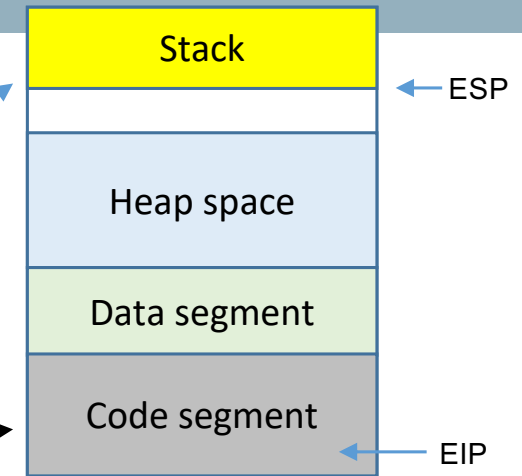
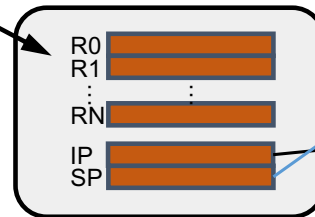
- Timer interrupt moves the process execution from user stack to its kernel stack
 - The transfer from user to kernel stack happens similar to syscall (except that it does not mandatorily needs EAX as in case of syscall)
- User context is saved on the bottom of the kernel stack and interrupt handler is invoked
- The handler will call `yield()` method as it was a timer interrupt
- The **yield** will invoke a call to the CPU scheduler, which will: a) save the kernel context of the user process at the top of the kernel stack, b) load the context of the CPU scheduler from the corresponding stack, and c) jump to the CPU scheduler stack where it will call the `schedule()` routine
- Each CPU has its own CPU scheduler process that runs on a separate kernel stack than that of the process it was originally executing

The Context Switch – switch(OldCtx, NewCtx)

- Typical steps for calling a context switch on x86
 - Load the parameters passed to the context switching routine in registers
 - Parameters would be the context of current process that has to be saved, and the context of the scheduler process that has to be restored

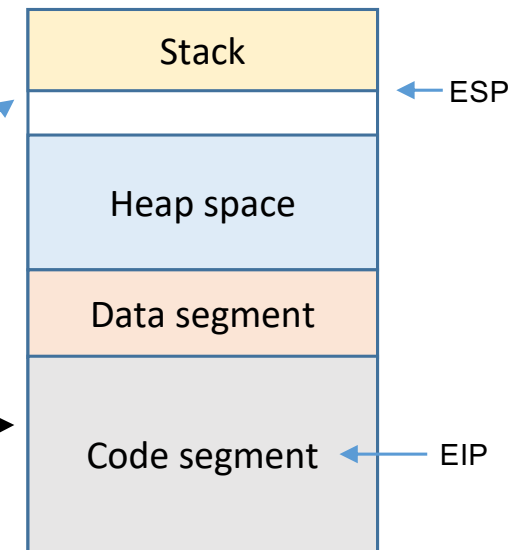
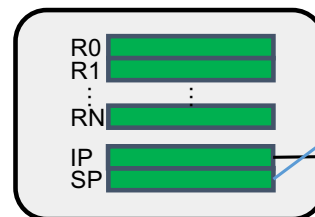
Hardware
Registers

Core-A



Context switching
from Process-1 to
Process-2

Core-A

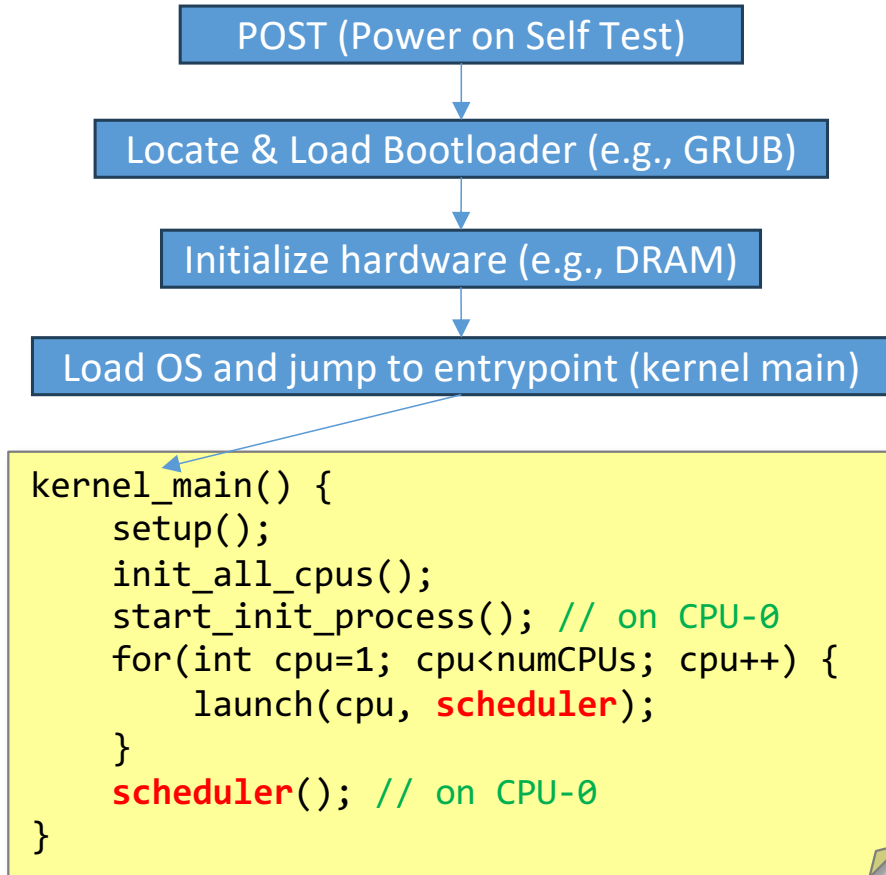


The Juggling API: scheduler

- **Basic questions about the design**

- How many scheduler processes should be there in an OS?
- How many copies of process table should be there in the OS?
 - Process table is an array of PCB
- Can there be any race on the process table accesses?
 - How to resolve?

Who Calls the scheduler



- OS is booted with the help of another program called bootloader
 - Reads the kernel ELF executable
 - Loads the program segments
 - Jump to the entry point (`kernel_main`)
 - Call never returns to bootloader (similar to `exec`)
- Kernel main
 - Complete the basic setup (initializing CPUs, memory, disk, registers, etc.)
 - Launch the first user process (?)
 - Launch one scheduler process on each CPU that will eventually pick a process from the process table and start executing it until it gives up the CPU (see case-1 and case-2 in previous slides)

The Juggling API: scheduler Pseudocode

```
void scheduler() {  
    while(true) {  
        foreach(Process p: (process_table)) {  
  
            swtch(scheduler_process, p);  
            // p is done for now..  
        }  
    }  
}
```

- Scheduler job is to simply pick a process from the process table
- Context switch of the scheduler process with the process picked from the process table
- Execute the process from the point it was previously stopped
- When the control will be passed back to scheduler from this process?

The Juggling API: scheduler Pseudocode

```
void scheduler() {
    while(true) {

        foreach(Process p: (process_table)) {
            if(p->state != READY) {
                continue;
            }
            p->state = RUNNING;

            switch(scheduler_process, p);
            // p is done for now..
        }
    }
}
```

- The scheduler should pay attention that the process picked up from the process table is in READY state (should not be in WAITING state)
- It should change the state of this process from READY to RUNNING
- When control is passed back to the scheduler from this process, its state would have changed to either READY or WAITING

The Juggling API: scheduler Pseudocode

```

void scheduler() {
    while(true) {
        lock(process_table);
        foreach(Process p: (process_table)) {
            if(p->state != READY) {
                continue;
            }
            p->state = RUNNING;

            switch(scheduler_process, p);
            // p is done for now..
        }
        unlock(process_table);
    }
}

```

Any issues here??

- There is a single process table on a system irrespective of the number of CPUs
- Proper locking mechanism must be ensured to avoid any race condition on the process table by the scheduler processes running at other CPUs
- Modern OS may use scalable implementations of locking to improve performance

The Juggling API: scheduler Pseudocode

```

void scheduler() {
    while(true) {
        lock(process_table);
        foreach(Process p: (process_table)) {
            if(p->state != READY) {
                continue;
            }
            p->state = RUNNING;
            unlock(process_table);
            switch(scheduler_process, p);
            // p is done for now..
            lock(process_table);
        }
        unlock(process_table);
    }
}

```

What would happen if
unlock isn't called?

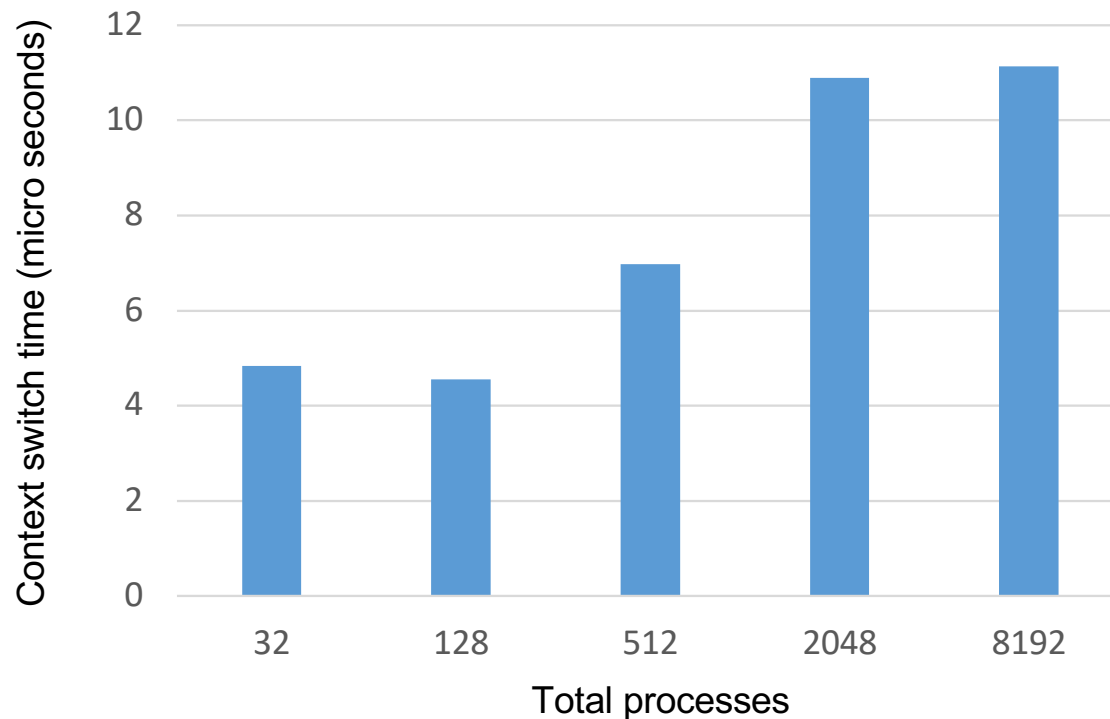
- There is a single process table on a system irrespective of the number of CPUs
- Proper locking mechanism must be ensured to avoid any race condition on the process table by the scheduler processes running at other CPUs
- Modern OS may use scalable implementations of locking to improve performance

The Juggling API: scheduler Pseudocode

```
void scheduler() {
    while(true) {
        lock(process_table);
        foreach(Process p: scheduling_algorithm(process_table)) {
            if(p->state != READY) {
                continue;
            }
            p->state = RUNNING;
            unlock(process_table);
            switch(scheduler_process, p);
            // p is done for now..
            lock(process_table);
        }
        unlock(process_table);
    }
}
```

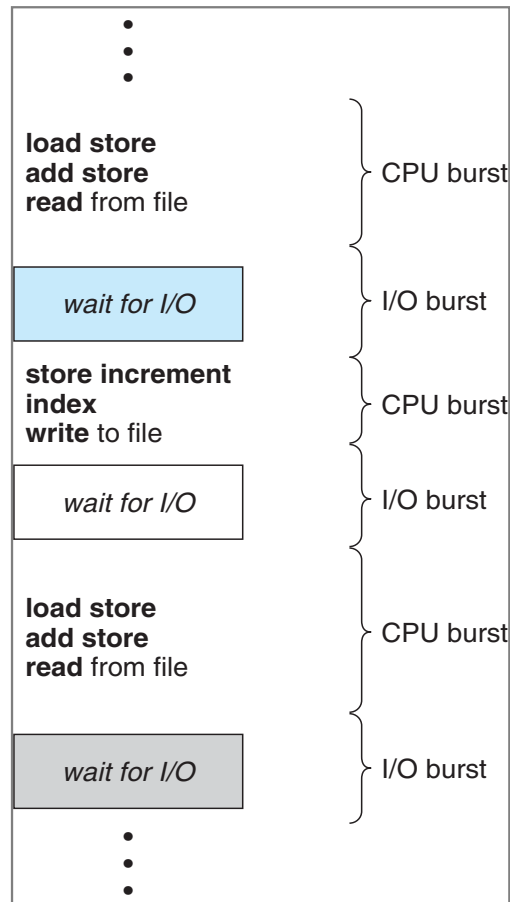
- Process scheduling algorithm plays an important role in the design of operating system
- Different algorithms are chosen according to the need

Context Switch: Cost?



- Context switch overhead measured on an AMD EPYC 32-core processor running Ubuntu 18.04.3 LTS
 - Data generated using Imbench benchmark (`./lat_ctx -s 0 32 128 512 2048 8192`)
- Overheads
 - Timer interrupt latency
 - Saving restoring context
 - Process scheduling
 - Loss in cache locality

CPU and I/O Bursts



- Process execution model: programs alternate between bursts of CPU and I/O
 - Program typically uses the CPU for some period of time, then does I/O, then uses CPU again
 - Each scheduling algorithm is about which process to give to the CPU for use by its next CPU burst
 - With time slicing, process may be forced to give up CPU before finishing current CPU burst

Key Scheduling Goals

- **Response Time**

- What user sees: time to echo a keystroke in editor

- **Throughput**

- Total operations per second
 - Affected by the overheads of context switching

- **Fairness**

- Conflicts with the response time

- **Starvation**

- Due to improper resource allocation policy

Reference Material

- <https://pdos.csail.mit.edu/6.828/2018/xv6/book-rev11.pdf>
 - You may read the chapter-5 just to have a high-level understanding of scheduling and context switch in a simple OS
 - You can avoid referring to the xv6 codebase as we will not have any assignment on reading/modifying xv6 code

Next Lecture

- Process scheduling policies